

FindIt: Campus Lost and Found Portal

Final Year Project Report

Submitted by: Group 07

A. Perera · B. Silva · C. Fernando · D. Jayasinghe

Faculty of Computing · [Your University]

Supervisor: [Supervisor name]

September 2026

WRITING TIP This is a **sample report** by ProjectMentor. Gold boxes like this one explain *why* it is written this way. Remove them from your own report. All numbers are sample data.

Abstract

Students often lose personal items on campus, and there is no single place to report or find them. This report presents FindIt, a web and mobile system that lets students report lost and found items and matches them automatically. The system was built with Flutter, ASP.NET Core and PostgreSQL, and a rule-based matching engine was developed to link similar reports. A four-week pilot was carried out with 120 students. The average time to recover an item fell from 14 days to 3 days. The results show that a central, automated system can return more lost items to their owners.

WRITING TIP The abstract is a **short summary of the whole report** (150-250 words). Use **present tense** for what the report does ("This report *presents...*") and **past tense** for what you did ("A pilot *was carried out*"). Write it last, after everything else is finished.

Table of contents

1. Introduction
 2. Background and related work
 3. Methodology
 4. Implementation
 5. Testing and results
 6. Discussion
 7. Conclusion and future work
- References

1. Introduction

1.1 Problem

Universities have thousands of students moving between buildings every day, and personal items are often lost. Today, students post about lost items in chat groups or on notice boards. These posts are hard to search, and owners and finders rarely see each other in time. As a result, many items are never returned.

1.2 Aim and objectives

The aim of this project is to build a central system that helps students get lost items back. The objectives are to:

(1) allow a student to report an item in under 30 seconds; (2) suggest likely matches automatically; and (3) reduce the time it takes to return an item to its owner.

WRITING TIP Use **present tense** for facts that are still true ("Universities *have* thousands of students"). Objectives should be **measurable** - you will check them again in the results section.

1.3 Report structure

Section 2 reviews existing approaches. Section 3 describes the methodology, and Section 4 explains the implementation. Section 5 presents the results, which are discussed in Section 6. Section 7 concludes the report.

2. Background and related work

Software projects benefit from a clear process that moves from requirements to design, implementation and testing [1], [2]. Existing lost-and-found approaches on campus are mostly manual: a physical office, notice boards, and social media groups. These methods depend on people checking them often, and they cannot link similar reports together. FindIt addresses this gap by combining a central database with automatic matching.

WRITING TIP When you mention other work, **cite it** with a number in square brackets, e.g. [1], and list it in the References. Use **past tense or present perfect** for what others did ("Earlier systems *focused on*..." / "Researchers *have shown*...").

3. Methodology

An agile approach was followed, with four two-week sprints. Requirements were collected through a short online survey and two group interviews with students. The system was designed as three parts: a Flutter mobile app, an ASP.NET Core web API [3], and a PostgreSQL database [5]. A rule-based matching engine was chosen because it is simple to test and explain.

WRITING TIP The methodology describes **what you did**, so it uses **past tense**. The **passive voice** ("Requirements *were collected*") is common here because the action matters more than who did it. Passive voice uses **was / were + past participle** (collected, designed, chosen).

4. Implementation

The mobile app was developed with Flutter [4] so that one code base could run on Android and iOS. Students can upload a photo, select a category, and pin the place where the item was lost or found. The web API exposes REST endpoints for reports, matches and claims. Each new report is compared with open reports of the opposite type; a match score is calculated from category, location and time, and matches above a threshold are sent to both students.

To keep data safe, students sign in with their university email, and an owner must answer a check question before an item is handed over.

WRITING TIP Explain **how** the system works and **why** you made each choice. Refer to diagrams and screenshots as "Figure 1", "Figure 2" and always explain them in the text - never drop a figure in without a sentence about it.

5. Testing and results

Unit and integration tests were written for the API, and a four-week pilot was carried out with 120 students. Table 1 compares the main measures before and after FindIt was introduced.

Table 1. Pilot results before and after FindIt (sample data)

Measure	Before FindIt	With FindIt
Average days to recover an item	14	3
Items returned to owners	31%	72%
Average time to report an item	n/a	26 seconds

The average time to recover an item fell from 14 days to 3 days, and the share of items returned to owners rose from 31% to 72%. Students reported items in 26 seconds on average, which meets the first objective.

WRITING TIP Table captions go **above** the table; figure captions go **below** the figure. Number them (Table 1, Figure 1) and mention each one in the text. Results use **past tense** and state the facts only - save your opinions for the Discussion.

6. Discussion

The results suggest that a central system with automatic matching can return items much faster than notice boards and chat groups. The largest gain came from instant alerts: students no longer needed to check for new posts. However, the pilot was short and involved only one faculty, so the results may not apply to the whole university. In addition, the rule-based

matching engine missed some items that were described in very different words.

WRITING TIP The discussion **explains what the results mean**. Use **present tense** for your interpretation ("The results *suggest*...") and be honest about **limitations** - examiners respect this.

7. Conclusion and future work

This project developed FindIt, a lost-and-found system for university students. All three objectives were met: reporting is fast, matches are suggested automatically, and items were returned in less time. In future, the system will be launched across the whole campus, and image-based matching will be added to improve accuracy.

WRITING TIP The conclusion **summarises** (past tense: "This project *developed*...") and links back to your objectives. Do not add new results here. Keep it short - one or two paragraphs.

References

- [1] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2020.
- [3] Microsoft, "ASP.NET Core documentation." [Online]. Available: <https://learn.microsoft.com/aspnet/core>
- [4] Google, "Flutter documentation." [Online]. Available: <https://docs.flutter.dev>
- [5] PostgreSQL Global Development Group, "PostgreSQL documentation." [Online]. Available: <https://www.postgresql.org/docs/>

WRITING TIP This list uses **IEEE style**: sources are numbered in the order you first cite them. Whatever style your university asks for (IEEE, APA, Harvard), use it **consistently** for every reference.